

Student Registration System: Project Report

1. Goal of the Project

The goal of this project is to build a robust and reliable **Student Registration System** backend. The system allows administrators and front-end applications to manage student records efficiently. It provides complete Create, Read, Update, and Delete (CRUD) capabilities for student entities, tracking essential data such as their name, contact email, enrollment year, and birth year.

2. Architecture

The project is built using the **Django Web Framework** following a **RESTful API Architecture**.

- **Backend Framework:** Python / Django
- **Database Model:** Uses Django's Object-Relational Mapping (ORM) to manage a **Student** entity.
- **Routing & Controllers:** The application cleanly separates concerns. **urls.py** manages endpoint routing, **views.py** acts as a dispatcher based on HTTP methods (GET, POST, PATCH, DELETE), and **controllers.py** handles the core business logic, database queries, and JSON serialization.
- **Data Exchange:** All client-server communication is handled via JSON (**JsonResponse**), adhering to REST principles without requiring traditional HTML template rendering.
- **Security Context:** API endpoints are designed to be stateless. Cross-Site Request Forgery (CSRF) protections are exempted for API usage, allowing seamless integration with external clients (like Postman or a React/Vue frontend).

3. Routes Implemented

The application exposes a standard set of REST API endpoints:

Action	HTTP Method	Endpoint	Description
List Students	GET	/api/students	Retrieves a full list of registered students in JSON format.
Create Student	POST	/api/students	Creates a new student record. Requires a JSON payload with name , email , password , enrollment_year , and birth_year .
Update Student	PATCH	/api/students/<id>	Partially updates an existing student's data using their unique id . Only fields provided in the JSON body are updated.
Delete Student	DELETE	/api/students/<id>	Deletes the student record matching the provided id .

4. Testing & Validation (Screenshots)

The API endpoints were thoroughly tested using Postman to validate the CRUD lifecycle. Below is the compiled evidence of the tests executed against the server.

[!NOTE] The server responds with appropriate HTTP status codes and JSON acknowledgment messages for every operation.

```
![Test Evidence 1](/home/mac/.gemini/antigravity/brain/7a471c4d-fe97-4253-99f0-a2eab5cc4257/artifacts/Screenshot%20From%202026-05-12%2012-04-16.png)
<!-- slide -->
![Test Evidence 2](/home/mac/.gemini/antigravity/brain/7a471c4d-fe97-4253-99f0-a2eab5cc4257/artifacts/Screenshot%20From%202026-05-12%2012-04-31.png)
<!-- slide -->
![Test Evidence 3](/home/mac/.gemini/antigravity/brain/7a471c4d-fe97-4253-99f0-a2eab5cc4257/artifacts/Screenshot%20From%202026-05-12%2012-14-23.png)
<!-- slide -->
![Test Evidence 4](/home/mac/.gemini/antigravity/brain/7a471c4d-fe97-4253-99f0-a2eab5cc4257/artifacts/Screenshot%20From%202026-05-12%2012-14-34.png)
<!-- slide -->
![Test Evidence 5](/home/mac/.gemini/antigravity/brain/7a471c4d-fe97-4253-99f0-a2eab5cc4257/artifacts/Screenshot%20From%202026-05-12%2012-14-56.png)
```